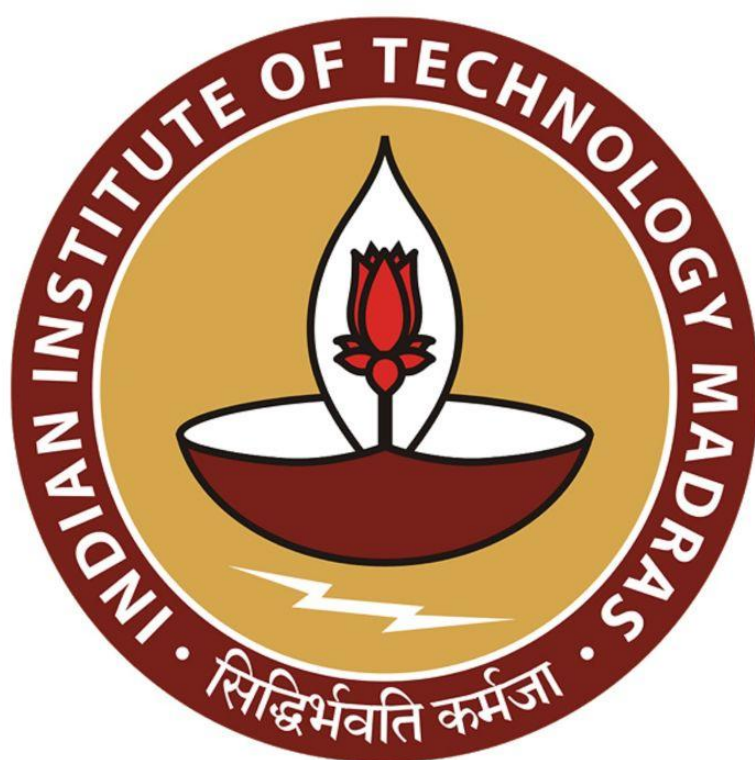
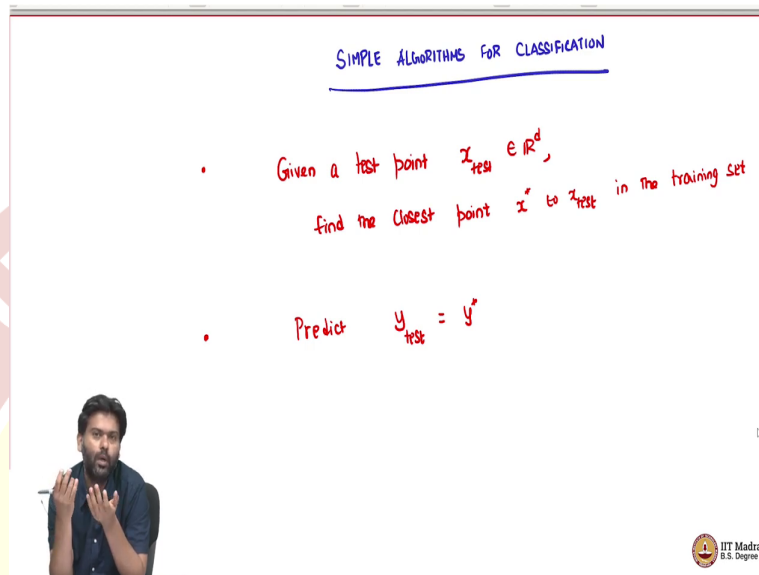




**IIT Madras**  
ONLINE DEGREE

**Machine Learning Techniques**  
**Professor Arun RajKumar**  
**Department of Computer Science and Engineering**  
**Indian Institute of Technology, Madras**  
**K-Nearest Neighbours**

(Refer Slide Time: 0:13)



Simple algorithms for classification. So, if you have given a thought I am sure you would have perhaps landed at the same algorithm or same idea as what I am about to say, you have a bunch of data points all those have corresponding labels. Now, I give you a new point. And now the question is, what is the label for this? If the new point is already in your training set, then yes you would tell the corresponding label of that point, so from the training set that is easy.

But if the new point is not in the training set, then how do you do this, that is the more I mean more natural question to ask because your test points are never going to be in the training set. So, what can you do? What is the simplest thing that you can do? You can look at who in the training set looks like our test data point and then we can ask that data point for the label, which means you look at the nearest point to the given data point, which the nearest point looks as similar to the test data point and so you look for the nearest data point in the training set to the test data point and then predict the test label as the label of the training point which is closest to the test point.

In other words, so what I am saying is given a test point let us call this some  $x_{\text{test}}$  which is in  $\mathbb{R}^d$  the algorithm is find the closest point  $x$ . Let us, let us say  $x^*$  to  $x_{\text{test}}$  closest to  $x_{\text{test}}$  in the training set. You found the closest point, what are you going to predict? Predict,  $y_{\text{test}}$  the

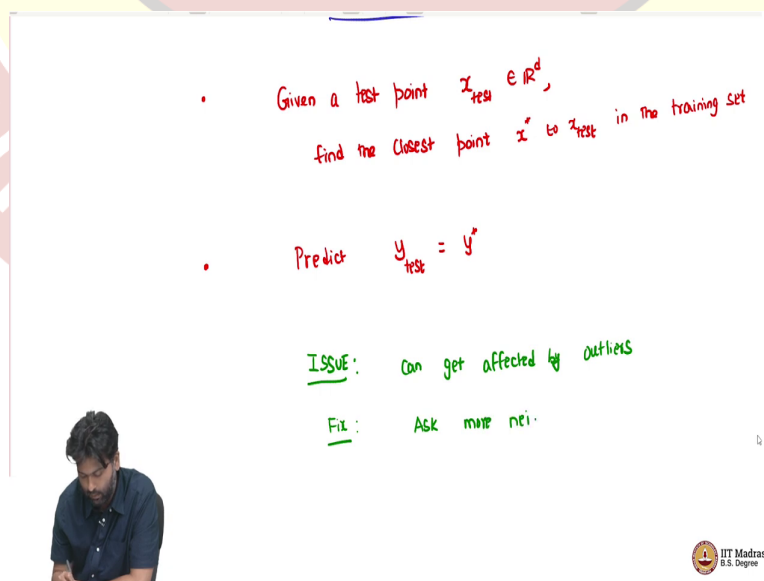
prediction for  $y_{\text{test}}$  is going to be simply  $y^*$ . So, the label of  $x^*$  that you have found. So, so this is the simplest algorithm I mean you do not know what the label is you ask your neighbor and then the neighbor says hey this is my label and then you predict the same label for your point as well.

Now, what might go wrong with this algorithm. So, is this a good algorithm? I mean we have spoken so much about classification and then NP hardness of the objective and so on. But then here is a super simple algorithm, is this a good algorithm, is something that we need to discuss next. What do you think, is this a good algorithm? If you think about it what might go wrong in this algorithm.

So, let us say your nearest point happens to be an outlier a point which should not have been in the in that position. So, maybe it is a +1 point which is among amidst the negative labeled points or a negative point which is amidst the +1 point. Maybe there is a human error that labeled it incorrectly, we do not know that. So, we because, we are just given a data set. We do not know how, what process went through when we created the data set and data sets are always noisy.

So, if your nearest neighbor happens to be an outlier then your algorithm does not know this and then it is just going to follow the nearest neighbors prediction and so it might also go wrong.

(Refer Slide Time: 4:06)



- Given a test point  $x_{\text{test}} \in \mathbb{R}^d$ , find the closest point  $x^*$  to  $x_{\text{test}}$  in the training set
- Predict  $y_{\text{test}} = y^*$

ISSUE: Can get affected by outliers

Fix: Ask more nei.

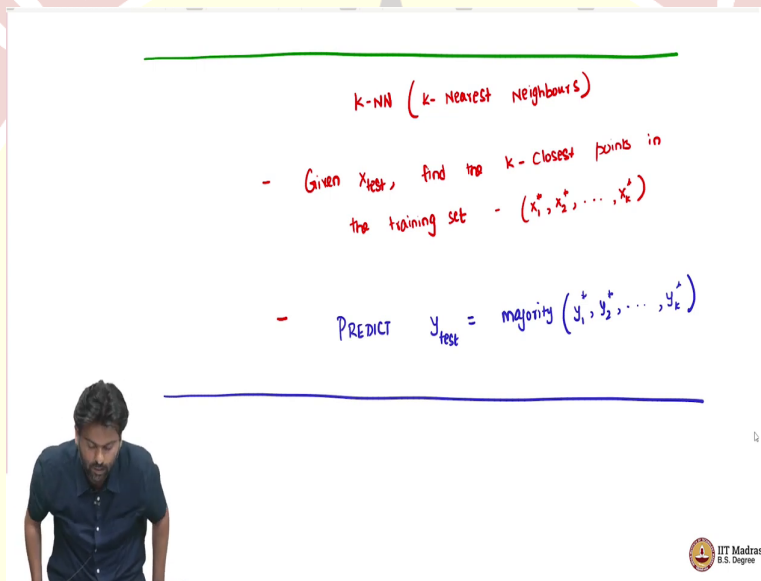


So, so the simple issue that might cause a problem with this algorithm is the following issue is, this algorithm can get affected by outliers and how do we fix this. What do you think, how

can we fix this? So, pause and think about this I will tell you one simple fix for this. So, your nearest neighbor is an outlier that is the problem. So, then do not ask just the nearest neighbor get opinion from more than one person. So, ask  $k$  different neighbors and then will take the majority. The hope is that if you ask  $k$  different neighbors their opinion not all of them or not majority of them are going to be outliers. So, if the majority of them are going to be outliers, then they are not really outliers. So, then maybe they are supposed to be there.

So, you can ask the majority, you can ask  $k$  neighbors and then take a majority vote and that would be your prediction. So, this is a simple fix ask more neighbors that is it.

(Refer Slide Time: 5:28)



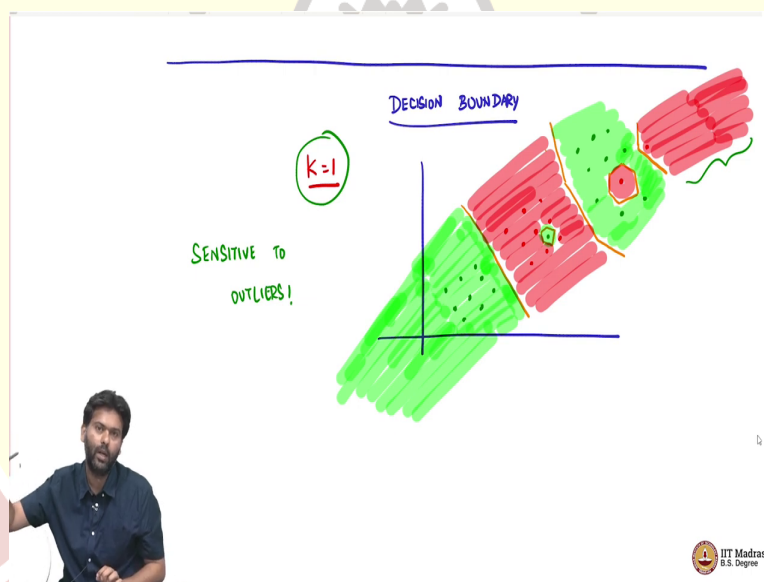
So, that will give us our first simple algorithm that we will see for binary classification which is the algorithm called as K-NN which stands for  $k$  nearest neighbors. So, what is the algorithm? Given  $x_{test}$ , find the  $k$  not just a single but then  $k$  closest points in the training set. And let us call these  $x_1^*$ ,  $x_2^*$ ,  $\dots$ ,  $x_k^*$ , let us say these are the  $k$  closest point to  $x_{test}$  of course it depends on  $x_{test}$ . Now, how are you going to predict? Your prediction is going to be predict  $y_{test}$  is simply going to be the majority vote of the corresponding labels of these nearest neighbors. So,  $y_1^*$ ,  $y_2^*$ ,  $\dots$ ,  $y_k^*$ . So, this is your first algorithm that we will see for our for binary classification, it is a super simple algorithm.

So, you just look at your neighbors and then follow whatever the majority says. So, this is this algorithm seems pretty easy and pretty straightforward. Now, there are certain things we need to think about this algorithm, we have put down this algorithm I mean if this algorithm was solved the entire problem then the course would end here in some sense. So, we have to

always ask, what can go wrong with any algorithm that we put down and that is that is very important to know. Because algorithms make assumptions and we should we have to be very clear what these assumptions are, and what might go wrong.

So, or in some sense in some cases is it practical to have such an algorithm, we might give a great algorithm but then if you are unable to run it at the end of the day if your code is going to take forever to run that is also an issue. You should think about all these questions and let us let us start by asking each of these questions. So, the first question we ask is, so there is a parameter that you have to supply as input to this algorithm which the data does not tell you, so directly so this parameter is  $k$ . So, how many neighbors should you look for, we it is not immediately obvious by just looking at this algorithm. We are saying we just ask  $k$  nearest neighbors.

(Refer Slide Time: 8:13)



So, now what is it, what is this  $k$ ? So, how does  $k$  affect our problem? This is something that we need to understand, for that we need to look at what is called as a decision boundary of our algorithms and this decision boundary is something to keep in mind not just for this algorithm for every algorithm that you will see in this course, ask yourself what is the decision boundary meaning in our input space where are, where is the algorithm trying to classify as  $+1$  and where the algorithm is trying to classify as  $-1$ . So, we need to understand that very very clearly. Once we understand that it will give us some insights.

So, let us see what the decision bound means let us look at the case where  $k=1$ . So, now we are still in let us say  $R^2$  two dimensional data maybe we have a bunch of points like this,

always I am going to think of green as negative and red as positive, maybe there are some points here, maybe there are more greens on this side, and maybe there is a red here, maybe there is a small red here, maybe there is a green here. So, maybe this is our data set.

Now, how is the decision boundary going to look like, for what is the decision boundary for every point in the two dimensional space I need to think of, what is the decision the algorithm would give me if that point was the test data point. So, every any point in the two dimensional space could be a potential test point. Now, you can ask where what does the algorithm predict? It is going to predict based on the data set that we that we have, so what what would the algorithm predict?

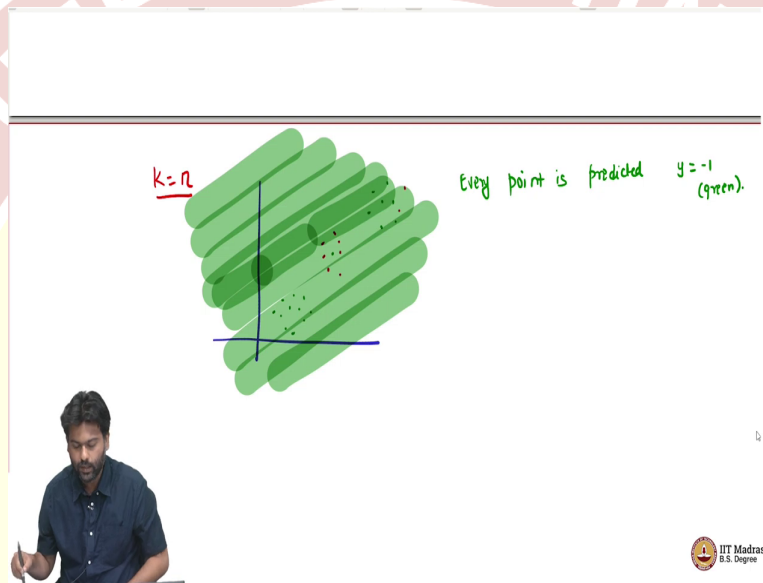
Now, if you plot the decision boundary for this algorithm. So, so all the points on the bottom part is in green. So, you might get a decision boundary something like this such that everybody on this side is green. So, because any point here in the green shaded region when I say this I mean this is an infinite region. So, all these points. So, again of course it keeps extending infinitely. All these points are green because they are closer to a green point  $k = 1$  remember, they are closer to a green point than red point.

Similarly, you might get something similar on the other side also you might get something like this maybe slightly tilt based on how the data points themselves are, maybe something like this. So, the, in between region is predominantly red except this small piece here. So, this region here now is a hole which is a green hole because there is this green point which is closer than the red points. Similarly, there is a slightly bigger hole here something like this maybe, so maybe this is a hole where it is going to be classified as red and because of this point, this one single red point here you might have something like this. So, what would happen is, these regions are all green, and now these regions this is all red from here on everything is red on the other side.

Everybody in between seems red, all this would be red except for this hole, except for this hole which is going to be green. Similarly, on the other side except for this hole which is going to be red. This is going to be your decision boundary. So, which means that any point, if it is in the green region is going to be predicted +1 and if it is in the -1, if it is the red region it is going to be predicted +1. The problem is there are these holes. So,  $k = 1$  this was the problem we spoke about earlier, there are these holes which  $k = 1$  has. So, that is that is the sensitivity to outliers part that I spoke about. So, sorry about this.

So, it is sensitive to outliers and that is the issue. What is sensitive to outliers? The fact that if you use  $k=1$  it becomes sensitive to outliers because these holes are found by these outliers. So, and if the outlier is on one end then the whole part here you are going to predict as red whereas it should have potentially been green. So, ideally it looks like the real structure is that there are two green parts and then the middle part is red, whereas we are now dividing it into four parts because there was an outlier at the end on the other side. So,  $k=1$  is clearly a bad idea.

(Refer Slide Time: 13:49)



Now, what happens if let us say we again take the same data set I will quickly redraw the data set, again you have a bunch of points here, bunch of points here, and a bunch of points here green ones, and then there is a green here, and then there are a bunch of reds here. Now, let us ask the question, what happens if  $k=n$ ? I think this is a question that you should think about yourself, if the number of neighbors that you are asking opinion for equals  $n$  the number of data points then how would the decision boundary look like.

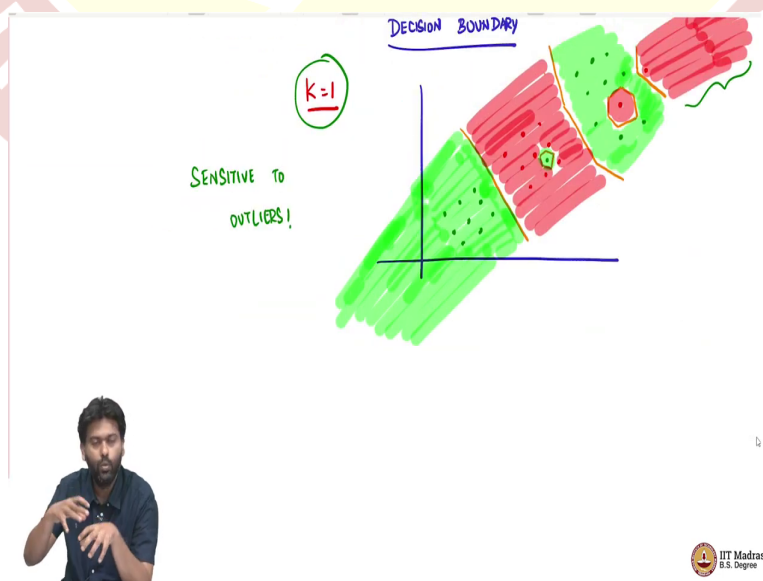
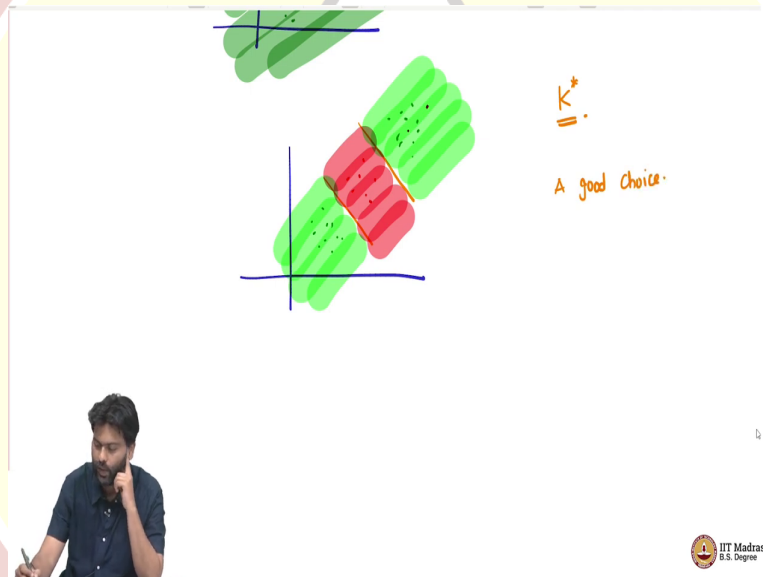
If for every data point, if you are asking everybody their opinion and then you are going to look at the majority. So, everybody every data point test at a point is going to be predicted simply the majority vote of your training data. So, because all everybody in the training data has gives an opinion and then you will take the majority and that is the same majority that will apply for any test data point.

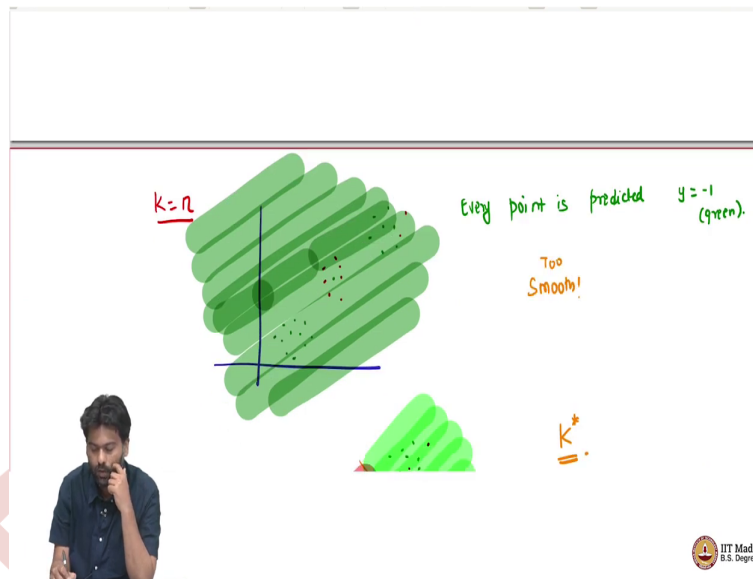
So, the decision boundary in this case is simply, let us say in this case if there are more greens than reds that is what that is how this looks like because there are two regions of greens and

one region of red. So, what would really happen is this decision boundary would simply be green everywhere. So, every region the whole of  $R_2$  is going to be predicted as green. So, I mean there is no gaps here. So, so what would happen is, if the whole of  $R_2$  is predicted green. So, any point or rather every point is predicted  $y = -1$  green which is also clearly a bad idea.

So, asking too few people leads to outliers problem asking too many people is always a bad idea. So, you should never ask too many people for opinions because they are just going to confuse you. So, and then you are simply going to always go wrong here as well. So, you have to ask the number of people, that is the that is the idea. So, there is some there is going to be some  $k^*$ . So, which will give us what we want.

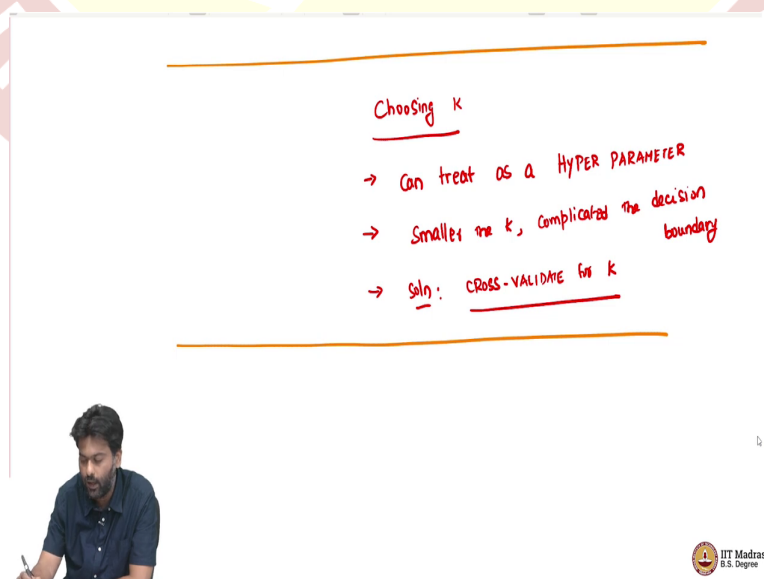
(Refer Slide Time: 16:14)





So, what is, what do we want really is you have these points like this and then you have these points of course you have outliers here, here and so on, yet you want to get this region. So, this is the region that you really want to achieve you need green on this side, you need green on this side which means the outliers on either side should not really matter, and you need red here, this is the essential structure in data. And there will be some  $k^*$  for which this will happen. So, if you ask the number of neighbors then this will you will get this. But the question is, how do we find this number of neighbors? So, so on one side it is too rugged, so you have all these holes, on the other side you have too smooth. So, too smooth that is the problem with  $k=n$  but then  $k$  equals this is a good choice. So,  $k^*$  is a good choice. But how do we choose this? The way to choose this is...

(Refer Slide Time: 17:25)



So, now how let us let us talk a bit about how to choose  $k$ , choosing  $k$ ,  $k$  is a parameter that we supply to the algorithm and then we ask the algorithm to run. So, based on the  $k$  that you supply. So, you should treat  $k$  as a hyper parameter and we have discussed hyper parameter in the context of regression, the idea is exactly the same, something that it is not part of the algorithm but then it is passed into the algorithm is a hyper parameter and we note that smaller the  $k$  smaller the  $k$  complicated the decision boundary. So, we need to find a good  $k$ .

So, what you would do is, you can just cross validate for  $k$ . So, solution cross validate for  $k$ , which means try out different  $k$  and see which  $k$  performs on the held out cross or validation set. So, whichever  $k$  gives you the best accuracy in the validation set, that is the  $k$  that you should look at. So, on the hope is that such a  $k$  will only capture the structure and not the outliers and you will get the boundary like this for instance for example. So, the idea is cross validation that is the most important point. So, you can cross validate for  $k$ , so that problem we can solve.

So, we know how to find a good  $k$ , let us say. Now, we have put down an algorithm we have said that you can solve the problem of choosing  $k$  by cross validation. So, does it mean that we have solved the binary classification problem is  $k$  nearest neighbor the best algorithm or are there some issues with the algorithm.

(Refer Slide Time: 18:58)

ISSUES with K-NN

- Choosing a distance function
- PREDICTION is COMPUTATIONALLY EXPENSIVE
- NO MODEL IS LEARNED.
  - Cannot throw away data after "LEARNING"

12

IIIT Madras  
B.S. Degree

Let us discuss, if there are any issues with  $k$  nearest neighbors, issues with K-NN. what do you think are some issues with  $k$  nearest neighbors. There are several issues with  $k$  nearest neighbors, for example issues like is how do we compute distance. So, choosing a distance

function might be an issue. Why should the euclidean distance be the right way to measure a neighbor. So, maybe different data different features have different ways of measuring neighborhood. So, how to choose a distance function is a, is an issue but let us say as a domain expert you somehow know how to design a good distance function that works that would naturally work for your data set at hand. So, let us say even this problem can be solved.

So, maybe some features have to be measured differently, some features have to be measured in a (diff) using L2 distance and so on, let us say we know that. So, let us say even this problem can be solved. What is a more pertinent problem with the k nearest neighbors? The biggest problem and we will see an outward understanding of this problem first, which is which I will talk about and then we will go into the roots of this problem which will be which it should be much more deeper and troublesome. The biggest problem is that what happens during prediction time.

So, when you try to predict a new data point  $x_{\text{test}}$ , what do you do, what should you do, what should your algorithm do. It has to find out what are the top k neighbors of this data point and to do that if you have a billion data points. So, then you will have to do some kind of a sorting of the distance between the test point and each of these billion data points and then pick the top key. So, of course sorting is going to take its own sweet time. So, if you have billion, so then in usual sorting algorithm is going to take billion log, billion times, maybe you will do some kind of hashing and so on to make it faster, yet it is going to take a lot of computational time.

So, to find this the top k neighbors maybe you can use some properties of the distance function to reduce this computation, still it is going to scale with the number of data points that you have in your data set. So, the prediction is going to take computationally a lot of time. And if you want to predict the next data point you will have to do the same business again. So, to find the distances sort them take the top k and then look at the labels.

So, prediction is computationally expensive, this is the most important problem with k nearest neighbors. So, this is the in some sense the outward problem, that we see when we try to implement this algorithm we see that prediction is going to be computationally intensive. But the deeper reason though is, why is prediction computationally expensive? Prediction is computationally expensive because our algorithm or what we call as the k nearest neighbor

algorithm is not really learning anything at all. If you think about it, what is being learned, nothing is being learned really.

So, you have a data set and then given a new data point you need the entire data set to make the prediction for this new data point. There is nothing that you are really learning from this data set. So, which means you are not really extracting the most interesting information from this data set and just retaining that information and then using that to make predictions. In other words, this extracting the useful information is what we call as a model and there is no model that is being learned at all. And so, your prediction becomes computationally expensive.

You cannot simply throw away the data points once you have extracted out the useful information, nothing like that is happening in k nearest neighbors. You need the entire data set to make prediction for every single test point and that is a problem. So, that is the main fundamental problem with k nearest neighbors that no model is learned.

So, in other words we cannot throw away data after learning, I am putting learning in codes because there is nothing that we are really learning at all in k nearest neighbors. So, this is the major problem of k nearest neighbors though even if you, even if you choose k carefully, even if you have a distance function to work with, even if you are computationally you have the capability to do computation large computation, still the problem remains that in the spirit of learning this is not really doing learning at all.

So, a real I mean, I would call something as learning from data, if once you have learned the most important interesting information structure about the data you throw away the data you do not need the data anymore. The structure has been captured in a succinct model, in succinct way again going back to the compression analogy that we started with this whole course with, we are not extracting any compressed information from the data that is enough to make predictions, we need the entire data set and that is the biggest issue.

So, what we will see from now on, in fact all the algorithms that we will see in a 1 is going to have the property that you are going to extract a model out of the data. Now, what kind of models you extract depends upon what assumptions you make about the data. So, and what kind of and that will lead us to different algorithms as we will see, and what we are going to see next is a different algorithm, which is still a simple algorithm a very very popular algorithm. But then it is not going to use the entire data set while prediction time, it is going

to extract a nice interpretable model out of this data set and this algorithm we will see next.

Thank you.

